Module 8: Portfolio Project

Start Assignment

- Points 300
- Submitting a file upload

Portfolio Project (300 Points)

Important! Read First

Choose one of the following two assignments to complete this week. Do not do both assignments. Identify your assignment choice in the title of your submission.

Option #1: Working with a Generative Adversarial Network

Part 1 (Research Write-up):

Research and analyze the use of generative adversarial networks in industry and in applications. Your use cases should be uniquely distinct and should cover multiple areas of industry. Ensure that your paper meeting the following guidelines:

- Identify at least 4 pertinent use cases in which this model is used and the benefit of using it in each.
- Your paper should be a maximum of 4 pages and include at least 3 scholarly references in APA format. Ensure that your assignment is formatted according to the CSU Global Writing Center. You can easily access the Writing Center by clicking on the tab in the course navigation panel.

Part 2 (Programming Implementation):

Generative Adversarial Networks (GANs) are a powerful class of neural networks that are used for unsupervised learning. It was developed and introduced by Ian J. Goodfellow in 2014. GANs are basically made up of two competing neural network models, which are able to analyze, capture, and copy the variations within a dataset.

Generative Adversarial Networks (GANs) can be broken down into three parts:

1. Generative: To learn a generative model, which describes how data is generated in terms of a probabilistic model.
2. Adversarial: The training of a model is done in an adversarial setting.
3. Networks: Use of deep neural networks as the artificial intelligence (AI) algorithms for training purposes.

In GANs, there is a **Generator** and a **Discriminator**. The Generator generates fake samples of data (an image, audio, etc.) and tries to fool the Discriminator. The Discriminator, on the other hand, tries to distinguish between the real and fake samples. The Generator and the Discriminator are both neural networks, and they both run in competition with each other in the training phase. The steps are repeated several times, and after each repetition, the Generator and Discriminator get better and better in their respective jobs.

Training a GAN has two parts:

A: The Discriminator is trained while the Generator is idle. In this phase, the network is only forward propagated and no back-propagation is done. The Discriminator is trained on real data for n epochs to see if it can correctly predict them as real. Also, in this phase, the Discriminator is trained on the fake generated data from the Generator to see if it can correctly predict them as fake.

B: The Generator is trained while the Discriminator is idle. After the Discriminator is trained by the generated fake data of the Generator, we can get its predictions and use the results to train the Generator and improve from the previous state to try and fool the Discriminator.

The above method is repeated for a few epochs, and then, we manually check the fake data if it seems genuine. If it seems acceptable, then the training is stopped; otherwise, it’s allowed to continue for a few more epochs.

Sample Python code implementing a Generative Adversarial Network:

```
# importing the necessary libraries and the MNIST dataset

import tensorflow as tf

import numpy as np

import matplotlib.pyplot as plt

from tensorflow.examples.tutorials.mnist import input_data

mnist = input_data.read_data_sets("MNIST_data")

# defining functions for the two networks.
```

```
# Both the networks have two hidden layers
# and an output layer, which are densely or
# fully connected layers defining the
# Generator network function
def generator(z, reuse = None):
        with tf.variable_scope('gen', reuse = reuse):
                hidden1 = tf.layers.dense(inputs = z, units = 128,
                                                        activation = tf.nn.leaky_relu)

                hidden2 = tf.layers.dense(inputs = hidden1,
                units = 128, activation = tf.nn.leaky_relu)

                output = tf.layers.dense(inputs = hidden2,
                        units = 784, activation = tf.nn.tanh)
                return output
# defining the Discriminator network function
def discriminator(X, reuse = None):
        with tf.variable_scope('dis', reuse = reuse):
                hidden1 = tf.layers.dense(inputs = X, units = 128,
                                                        activation = tf.nn.leaky_relu)

                hidden2 = tf.layers.dense(inputs = hidden1,
                        units = 128, activation = tf.nn.leaky_relu)

                logits = tf.layers.dense(hidden2, units = 1)
                output = tf.sigmoid(logits)
                return output, logits
# creating placeholders for the outputs
```

```
tf.reset_default_graph()

real_images = tf.placeholder(tf.float32, shape =[None, 784])

z = tf.placeholder(tf.float32, shape =[None, 100])

G = generator(z)

D_output_real, D_logits_real = discriminator(real_images)

D_output_fake, D_logits_fake = discriminator(G, reuse = True)

# defining the loss function

def loss_func(logits_in, labels_in):

        return tf.reduce_mean(tf.nn.sigmoid_cross_entropy_with_logits(

                                                logits = logits_in, labels = labels_in))

# Smoothing for generalization

D_real_loss = loss_func(D_logits_real, tf.ones_like(D_logits_real)*0.9)

D_fake_loss = loss_func(D_logits_fake, tf.zeros_like(D_logits_real))

D_loss = D_real_loss + D_fake_loss


G_loss = loss_func(D_logits_fake, tf.ones_like(D_logits_fake))

# defining the learning rate, batch size, and

# number of epochs and using the Adam optimizer

lr = 0.001 # learning rate

# Do this when multiple networks

# interact with each other

# returns all variables created(the two

# variable scopes) and makes trainable true

tvars = tf.trainable_variables()

d_vars =[var for var in tvars if 'dis' in var.name]

g_vars =[var for var in tvars if 'gen' in var.name]

D_trainer = tf.train.AdamOptimizer(lr).minimize(D_loss, var_list = d_vars)
```

```
G_trainer = tf.train.AdamOptimizer(lr).minimize(G_loss, var_list = g_vars)
batch_size = 100 # batch size
epochs = 500 # number of epochs. The higher the better the result
init = tf.global_variables_initializer()
# creating a session to train the networks
samples =[] # generator examples
with tf.Session() as sess:
        sess.run(init)
        for epoch in range(epochs):
                num_batches = mnist.train.num_examples//batch_size
                for i in range(num_batches):
                        batch = mnist.train.next_batch(batch_size)
                        batch_images = batch[0].reshape((batch_size, 784))
                        batch_images = batch_images * 2-1
                        batch_z = np.random.uniform(-1, 1, size =(batch_size, 100))
                        _= sess.run(D_trainer, feed_dict ={real_images:batch_images,
z:batch_z})
                        _= sess.run(G_trainer, feed_dict ={z:batch_z})

                print("on epoch{}".format(epoch))
                sample_z = np.random.uniform(-1, 1, size =(1, 100))
                gen_sample = sess.run(generator(z, reuse = True),
                                                                feed_dict =
{z:sample_z})

                samples.append(gen_sample)
# result after 0th epoch
plt.imshow(samples[0].reshape(28, 28))
```

```
# result after 499th epoch

plt.imshow(samples[49].reshape(28, 28))
```

The result after the 0^{th} epoch: (Note: The figures used in this problem specification were generated by the given Python code.)

Result after the 499^{th} epoch:

For this final Portfolio Project, you will build a GAN using the Keras library. (Keras library is a wrapper for low-level TensorFlow commands, and you will import this as a Python library.) The dataset used is the **CIFAR10 Image dataset,** which is preloaded into Keras. You can read about the dataset **here (https://keras.io/datasets/)** .

Step 1: Import the required Python libraries:

```
import numpy as np

import matplotlib.pyplot as plt

import keras

from keras.layers import Input, Dense, Reshape, Flatten, Dropout

from keras.layers import BatchNormalization, Activation, ZeroPadding2D

from keras.layers.advanced_activations import LeakyReLU

from keras.layers.convolutional import UpSampling2D, Conv2D

from keras.models import Sequential, Model

from keras.optimizers import Adam,SGD
```

Step 2: Load the data.

```
#Loading the CIFAR10 data

(X, y), (_, _) = keras.datasets.cifar10.load_data()


#Selecting a single class of images

#The number was randomly chosen and any number

#between 1 and 10 can be chosen
```

```
X = X[y.flatten() == 8]
```

Step 3: Define parameters to be used in later processes.

```
#Defining the Input shape
image_shape = (32, 32, 3)

latent_dimensions = 100
```

Step 4: Define a utility function to build the generator.

```
def build_generator():
    model = Sequential()
    #Building the input layer
    model.add(Dense(128 * 8 * 8, activation="relu",
                    input_dim=latent_dimensions))
    model.add(Reshape((8, 8, 128)))

    model.add(UpSampling2D())
    model.add(Conv2D(128, kernel_size=3, padding="same"))
    model.add(BatchNormalization(momentum=0.78))
    model.add(Activation("relu"))

    model.add(UpSampling2D())

    model.add(Conv2D(64, kernel_size=3, padding="same"))
    model.add(BatchNormalization(momentum=0.78))
```

```
    model.add(Activation("relu"))

    model.add(Conv2D(3, kernel_size=3, padding="same"))
    model.add(Activation("tanh"))

    #Generating the output image
    noise = Input(shape=(latent_dimensions,))
    image = model(noise)
    return Model(noise, image)
```

Step 5: Define a utility function to build the discriminator.

```
def build_discriminator():
    #Building the convolutional layers
    #to classify whether an image is real or fake
    model = Sequential()
    model.add(Conv2D(32, kernel_size=3, strides=2,
                     input_shape=image_shape, padding="same"))
    model.add(LeakyReLU(alpha=0.2))
    model.add(Dropout(0.25))

    model.add(Conv2D(64, kernel_size=3, strides=2, padding="same"))
    model.add(ZeroPadding2D(padding=((0,1),(0,1))))
    model.add(BatchNormalization(momentum=0.82))
    model.add(LeakyReLU(alpha=0.25))
    model.add(Dropout(0.25))

    model.add(Conv2D(128, kernel_size=3, strides=2, padding="same"))
```

```
    model.add(BatchNormalization(momentum=0.82))
    model.add(LeakyReLU(alpha=0.2))
    model.add(Dropout(0.25))

    model.add(Conv2D(256, kernel_size=3, strides=1, padding="same"))
    model.add(BatchNormalization(momentum=0.8))
    model.add(LeakyReLU(alpha=0.25))
    model.add(Dropout(0.25))

    #Building the output layer
    model.add(Flatten())
    model.add(Dense(1, activation='sigmoid'))

    image = Input(shape=image_shape)
    validity = model(image)

    return Model(image, validity)
```

Step 6: Define a utility function to display the generated images.

```
def display_images():
    r, c = 4,4
    noise = np.random.normal(0, 1, (r * c,latent_dimensions))
    generated_images = generator.predict(noise)

    #Scaling the generated images
    generated_images = 0.5 * generated_images + 0.5
```

```
    fig, axs = plt.subplots(r, c)
    count = 0
    for i in range(r):
        for j in range(c):
            axs[i,j].imshow(generated_images[count, :,:,])
            axs[i,j].axis('off')
            count += 1
    plt.show()
    plt.close()
```

Step 7: Build the GAN.

```
# Building and compiling the discriminator
discriminator = build_discriminator()
discriminator.compile(loss='binary_crossentropy',
                      optimizer=Adam(0.0002,0.5),
                     metrics=['accuracy'])
#Making the discriminator untrainable
#so that the generator can learn from fixed gradient
discriminator.trainable = False

# Building the generator
generator = build_generator()
#Defining the input for the generator
#and generating the images
z = Input(shape=(latent_dimensions,))
image = generator(z)
```

```
#Checking the validity of the generated image
valid = discriminator(image)

#Defining the combined model of the generator and the discriminator
combined_network = Model(z, valid)
combined_network.compile(loss='binary_crossentropy',
                         optimizer=Adam(0.0002,0.5))
```

Step 8: Train the network.

```
num_epochs=15000
batch_size=32
display_interval=2500
losses=[]

#Normalizing the input
X = (X / 127.5) - 1.

#Defining the Adversarial ground truths
valid = np.ones((batch_size, 1))

#Adding some noise
valid += 0.05 * np.random.random(valid.shape)
fake = np.zeros((batch_size, 1))
fake += 0.05 * np.random.random(fake.shape)

for epoch in range(num_epochs):
```

```
#Training the Discriminator

#Sampling a random half of images
index = np.random.randint(0, X.shape[0], batch_size)
images = X[index]

#Sampling noise and generating a batch of new images
noise = np.random.normal(0, 1, (batch_size, latent_dimensions))
generated_images = generator.predict(noise)

#Training the discriminator to detect more accurately
#whether a generated image is real or fake
discm_loss_real = discriminator.train_on_batch(images, valid)
discm_loss_fake = discriminator.train_on_batch(generated_images, fake)
discm_loss = 0.5 * np.add(discm_loss_real, discm_loss_fake)

#Training the generator

#Training the generator to generate images
#that pass the authenticity test
genr_loss = combined_network.train_on_batch(noise, valid)

#Tracking the progress
if epoch % display_interval == 0:
    display_images()
```